МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №5

з дисципліни «Функціональне програмування»

на тему: «Ліниві обчислення: генератори та itertools»

Варіант 6: «Потік фінансових транзакцій»

Виконав:
студент групи ФЕП-23
Корж Олексій

Перевірив:
проф. Свелеба С. А.

Львів – 2026

1. Мета роботи

Опанувати генераторні вирази (generator expressions), ключове слово yield та yield from, інтерфейс ітераторів та концепцію лінивих обчислень; навчитися працювати з нескінченними послідовностями без повного завантаження в оперативну пам’ять; освоїти будівельні блоки модуля itertools (accumulate, islice, cycle) та реалізувати ковзне вікно sliding_window, take та drop.

2. Теоретичні відомості

Ліниві обчислення (Lazy Evaluation): стратегія обчислень, за якої значення генерується лише в момент безпосередньої вимоги споживача. Дозволяє працювати з необмеженими обсягами даних із константним споживанням пам’яті O(1).

Генератори та yield: функції, що містять yield, повертають об’єкт-генератор, який зберігає стан виконання (стек, змінні) між послідовними викликами next(). yield from делегує генерацію підвиразам або вкладеним ітераторам.

Модуль itertools: стандартна бібліотека високоефективних ітераторних функцій. accumulate обчислює префіксні суми/накопичення, islice виконує зрізи ітераторів без копіювання, cycle нескінченно повторює послідовність.

Ковзне вікно (sliding window): алгоритмічний патерн аналізу часових рядів, реалізований через collections.deque(maxlen=k) для обчислення локальних агрегатів (сум, середніх).

3. Постановка завдання (Варіант 6)

Тема варіанта 6: Потік фінансових транзакцій.

Вхідний потік: Лінивий потік сум транзакцій (нескінченний генератор).

Основні завдання: Фільтрувати додатні операції через генераторні вирази, обчислювати накопичувальний баланс через itertools.accumulate, обчислювати перші N ковзних сум та ковзних середніх через sliding_window.

Обов’язкові інструменти: generator expression, accumulate, islice, sliding_window, take, drop. Обов’язкова перевірка суворої лінивості (відсутність зайвих обчислень).

4. Хід роботи та лістинг програми

Універсальні генератори потоків (streams.py)

```
from __future__ import annotations

from collections import deque
from collections.abc import Iterable, Iterator
from itertools import islice
from typing import TypeVar

T = TypeVar("T")


def take(n: int, it: Iterable[T]) -> Iterator[T]:
    if n < 0:
        raise ValueError("n must be non-negative")
    return islice(it, n)


def drop(n: int, it: Iterable[T]) -> Iterator[T]:
    if n < 0:
        raise ValueError("n must be non-negative")
    iterator = iter(it)
    for _ in range(n):
        try:
            next(iterator)
        except StopIteration:
            return
    yield from iterator


def sliding_window(it: Iterable[T], k: int) -> Iterator[tuple[T, ...]]:
    if k <= 0:
        raise ValueError("k must be greater than 0")
    iterator = iter(it)
    window: deque[T] = deque(islice(iterator, k), maxlen=k)
    if len(window) < k:
        return
    yield tuple(window)
    for elem in iterator:
        window.append(elem)
        yield tuple(window)


def fib_stream() -> Iterator[int]:
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
```

Фінансові операції варіанта 6 (transactions.py)

```
from __future__ import annotations

from collections.abc import Iterable, Iterator
from dataclasses import dataclass
from itertools import accumulate, cycle
from typing import TypeVar

from streams import sliding_window

T = TypeVar("T")


@dataclass(frozen=True, slots=True)
class Transaction:
    tx_id: int
    amount: float
    category: str


def infinite_transaction_stream(
    pattern: tuple[float, ...] = (100.0, -40.0, 250.0, -15.0, 50.0, -120.0, 300.0)
) -> Iterator[float]:
    yield from cycle(pattern)


def filter_positive(amounts: Iterable[float]) -> Iterator[float]:
    return (round(a, 2) for a in amounts if a > 0.0)


def cumulative_balance(
    amounts: Iterable[float], initial_balance: float = 0.0
) -> Iterator[float]:
    return (
        round(val, 2)
        for val in accumulate(
            amounts,
            lambda acc, x: acc + x,
            initial=initial_balance,
        )
    )


def moving_sums(amounts: Iterable[float], k: int) -> Iterator[float]:
    return (round(sum(w), 2) for w in sliding_window(amounts, k))


def moving_averages(amounts: Iterable[float], k: int) -> Iterator[float]:
    return (round(sum(w) / k, 2) for w in sliding_window(amounts, k))
```

Демонстраційний модуль (main.py)

```
from __future__ import annotations

from streams import drop, fib_stream, take
from transactions import (
    cumulative_balance,
    filter_positive,
    infinite_transaction_stream,
```

```
    moving_averages,
    moving_sums,
)


def main() -> None:
    raw_stream = infinite_transaction_stream()
    first_10_raw = list(take(10, raw_stream))
    print("Перші 10 транзакцій:", first_10_raw)

    pos_stream = filter_positive(infinite_transaction_stream())
    first_8_pos = list(take(8, pos_stream))
    print("Перші 8 додатних операцій:", first_8_pos)

    balance_stream = cumulative_balance(first_8_pos, initial_balance=500.0)
    print("Накопичувальний баланс:", list(balance_stream))

    window_size = 3
    sums_stream = moving_sums(first_8_pos, window_size)
    print(f"Перші ковзні суми (вікно {window_size}):", list(sums_stream))

    avg_stream = moving_averages(first_8_pos, window_size)
    print(f"Перші ковзні середні (вікно {window_size}):", list(avg_stream))

    dropped_pos = list(take(5, drop(3, filter_positive(infinite_transaction_stream()))))
    print("Пропуск перших 3 та вибір наступних 5 додатних:", dropped_pos)

    first_7_fib = list(take(7, fib_stream()))
    print("Лінивий стрім Фібоначчі (перші 7):", first_7_fib)


if __name__ == "__main__":
    main()
```

Тести універсальних функцій (tests/test_streams.py)

```
import pytest
from streams import drop, fib_stream, sliding_window, take


def test_take_basic() -> None:
    data = [10, 20, 30, 40, 50]
    assert list(take(3, data)) == [10, 20, 30]
    assert list(take(0, data)) == []
    assert list(take(10, data)) == [10, 20, 30, 40, 50]


def test_take_negative() -> None:
    with pytest.raises(ValueError):
        list(take(-1, [1, 2, 3]))


def test_drop_basic() -> None:
    data = [10, 20, 30, 40, 50]
    assert list(drop(2, data)) == [30, 40, 50]
    assert list(drop(0, data)) == [10, 20, 30, 40, 50]
    assert list(drop(10, data)) == []
```

```
def test_drop_negative() -> None:
    with pytest.raises(ValueError):
        list(drop(-1, [1, 2, 3]))


def test_sliding_window_basic() -> None:
    data = [1, 2, 3, 4, 5]
    windows = list(sliding_window(data, 3))
    assert windows == [(1, 2, 3), (2, 3, 4), (3, 4, 5)]


def test_sliding_window_edge_cases() -> None:
    assert list(sliding_window([], 3)) == []
    assert list(sliding_window([1, 2], 3)) == []
    with pytest.raises(ValueError):
        list(sliding_window([1, 2, 3], 0))
    with pytest.raises(ValueError):
        list(sliding_window([1, 2, 3], -2))


def test_fib_stream() -> None:
    assert list(take(8, fib_stream())) == [0, 1, 1, 2, 3, 5, 8, 13]
```

Тести фінансового конвеєра (tests/test_transactions.py)

```
from streams import take
from transactions import (
    cumulative_balance,
    filter_positive,
    infinite_transaction_stream,
    moving_averages,
    moving_sums,
)


def test_filter_positive(sample_amounts: list[float]) -> None:
    res = list(filter_positive(sample_amounts))
    assert res == [100.0, 200.0, 300.0, 150.0]


def test_cumulative_balance(sample_amounts: list[float]) -> None:
    res = list(cumulative_balance(sample_amounts, initial_balance=0.0))
    # [0.0, 100.0, 50.0, 250.0, 225.0, 525.0, 445.0, 595.0]
    assert res[0] == 0.0
    assert res[1] == 100.0
    assert res[2] == 50.0
    assert res[-1] == 595.0


def test_moving_sums_and_averages(sample_amounts: list[float]) -> None:
    positive = list(filter_positive(sample_amounts))
    # positive: [100.0, 200.0, 300.0, 150.0]
    sums = list(moving_sums(positive, 2))
    assert sums == [300.0, 500.0, 450.0]

    avgs = list(moving_averages(positive, 2))
    assert avgs == [150.0, 250.0, 225.0]
```

```
def test_infinite_stream() -> None:
    stream = infinite_transaction_stream((10.0, -5.0))
    sample = list(take(5, stream))
    assert sample == [10.0, -5.0, 10.0, -5.0, 10.0]
```

Тести лінивості обчислень (tests/test_laziness.py)

```
from collections.abc import Iterator

from streams import sliding_window, take
from transactions import filter_positive


def test_laziness_evaluated_only_requested() -> None:
    evaluations = 0

    def traced_stream() -> Iterator[float]:
        nonlocal evaluations
        val = 10.0
        while True:
            evaluations += 1
            yield val
            val += 10.0

    stream = traced_stream()
    result = list(take(4, stream))

    assert result == [10.0, 20.0, 30.0, 40.0]
    assert evaluations == 4


def test_filter_laziness() -> None:
    evaluations = 0

    def mixed_stream() -> Iterator[float]:
        nonlocal evaluations
        vals = [10.0, -5.0, 20.0, -15.0, 30.0, -25.0, 40.0, -35.0, 50.0]
        for v in vals:
            evaluations += 1
            yield v

    pos = filter_positive(mixed_stream())
    res = list(take(2, pos))

    assert res == [10.0, 20.0]
    assert evaluations == 3


def test_sliding_window_laziness() -> None:
    evaluations = 0

    def endless() -> Iterator[int]:
        nonlocal evaluations
        x = 1
        while True:
            evaluations += 1
            yield x
            x += 1
```

```
    windows = list(take(2, sliding_window(endless(), 3)))
    assert windows == [(1, 2, 3), (2, 3, 4)]
    assert evaluations == 4
```

5. Результати виконання та тестування

Запуск демонстраційної програми main.py

```
$ python main.py
Перші 10 транзакцій: [100.0, -40.0, 250.0, -15.0, 50.0, -120.0, 300.0, 100.0, -40.0, 250.0]
Перші 8 додатних операцій: [100.0, 250.0, 50.0, 300.0, 100.0, 250.0, 50.0, 300.0]
Накопичувальний баланс: [500.0, 600.0, 850.0, 900.0, 1200.0, 1300.0, 1550.0, 1600.0, 1900.0]
Перші ковзні суми (вікно 3): [400.0, 600.0, 450.0, 650.0, 400.0, 600.0]
Перші ковзні середні (вікно 3): [133.33, 200.0, 150.0, 216.67, 133.33, 200.0]
Пропуск перших 3 та вибір наступних 5 додатних: [300.0, 100.0, 250.0, 50.0, 300.0]
Лінивий стрім Фібоначчі (перші 7): [0, 1, 1, 2, 3, 5, 8]
```

Результати виконання тестів pytest

```
$ pytest -q
..............                                                           [100%]
14 passed in 0.54s
```

Статична перевірка типів mypy --strict

```
$ mypy --strict .
Success: no issues found in 7 source files
```

Перевірка форматування ruff та black

```
$ ruff check .
All checks passed!

$ black --check .
All done! ✨ 🍰 ✨
7 files would be left unchanged.
```

6. Висновки

У лабораторній роботі № 5 успішно реалізовано обробку нескінченних потоків фінансових транзакцій за допомогою лінивих генераторів та інструментів itertools. Забезпечено оптимальне використання пам’яті без повної матеріалізації потоків. Тести лінивості довели, що обчислюються суворо лише запитані елементи, усуваючи надлишкові накладні витрати.